CHANDIGARH UNIVERSITY

Department of Computer Applications

A PROJECT REPORT ON

PERSONAL EXPENSE TRACKER

Object Oriented Programming using C++

Unique Feature: **No-Spend Streak Challenge**

Submitted by	Suman
UID	25BCD10173
Branch	BCA (Data Science)
Section	25BCD3 (B)
Submitted to	Abhinay Sood
Institute	Chandigarh University

Academic Session 2025 - 2026

Declaration

I, **Suman**, a student of BCA (Data Science), Section 25BCD3 (B), University ID **25BCD10173**, Chandigarh University, hereby declare that the project entitled **"Personal Expense Tracker"** submitted to **Abhinay Sood** is my own work, carried out as part of the Object Oriented Programming course. The program has been written, compiled and tested by me in C++, and the unique feature **No-Spend Streak Challenge** has been designed and implemented as per the instructions given by the faculty.

The report has not been submitted to any other institute or university for any other purpose.

Date: _____

Signature: _____ (Suman, 25BCD10173)

Acknowledgement

I express my sincere gratitude to my faculty mentor **Abhinay Sood** for giving me the opportunity to build this project and for the clear instruction that every student must add one unique feature. This requirement pushed me to think beyond a basic menu-driven program and to apply the real principles of object oriented design.

I also thank Chandigarh University for the laboratory facilities, my classmates of section 25BCD3 (B) for their suggestions during testing, and my family for their constant support.

Table of Contents

No.	Chapter	Page
1	Abstract	4
2	Introduction	4
3	Objectives of the Project	5
4	Problem Statement and Scope	5
5	System Requirements	6
6	OOP Concepts Used	6
7	System Design and Class Description	8
8	UML Class Diagram	10
9	Unique Feature: No-Spend Streak Challenge	11
10	File Handling and Data Format	13
11	Source Code	14
12	Sample Output	18
13	Testing	19
14	Limitations and Future Scope	20
15	Conclusion and References	20

1. Abstract

The **Personal Expense Tracker** is a console based application written in C++ that helps a person record income, record daily expenses, search past records, view month-wise category summaries, and compare spending against a monthly budget. The project is built entirely around object oriented programming: abstraction through an abstract *Transaction* class, inheritance through *Expense* and *Income*, encapsulation of data in private and protected members, polymorphism through virtual functions, operator overloading in the *Date* class and file handling for permanent storage.

As required by the faculty, the project includes one unique feature. My feature is the **No-Spend Streak Challenge**: the user starts a challenge with a target number of days, and the program analyses the recorded expenses to find the days on which nothing was spent, the current streak, the best streak, an estimate of the money saved, a badge (Bronze, Silver, Gold, Platinum) and a monthly calendar. It turns saving money into a small game, which makes the tracker motivating and not just a record keeper.

2. Introduction

Most students and young professionals do not know where their money goes. A small chai, an online order, a bus ticket - each looks tiny, but at the end of the month the wallet is empty. Keeping a written diary is boring and easily forgotten. A simple program that stores every transaction and shows a summary solves part of this problem.

This project implements such a program using the C++ language and the principles of Object Oriented Programming (OOP). The application runs in the console, works on Windows, Linux and macOS, needs no internet and stores its data in plain text files so that the information is still available the next time the program is started.

Besides the standard functions of an expense tracker, the program focuses on behaviour: the unique No-Spend Streak Challenge encourages the user to avoid unnecessary spending, which is the real purpose of tracking expenses.

3. Objectives of the Project

- To design and implement a complete menu-driven expense tracker in C++ using object oriented principles.
- To record income and expenses with date, amount, category/source and a description.
- To save all data in files so that it is not lost when the program closes (persistence).
- To give month-wise summaries and a budget alert so that overspending is noticed immediately.
- To validate every input (dates, numbers, empty text) so that the program never crashes on wrong input.
- To add one original feature, the **No-Spend Streak Challenge**, that is not part of an ordinary tracker.
- To prepare proper documentation: UML class diagram, source code, sample output and this report.

4. Problem Statement and Scope

Problem statement: Develop an object oriented C++ program that allows a user to manage personal finances by adding, viewing, searching and deleting transactions, producing summaries and budget warnings, and that also contains one extra, unique feature chosen by the student.

Scope of the project:

- Single-user, console based application.
- Stores unlimited transactions in memory (limited only by RAM) and in a text file.
- Handles income and expenses in any currency (the program shows plain numbers).
- Date handling is done by the program itself, including leap years, so no external library is required.
- Out of scope: graphical interface, online banking sync, multi-user login (listed under future scope).

Why this project is useful

A student who sees that 40% of the month's money went to food delivery will change the habit. A user who is told "You have used 80% of your budget" on the 15th of the month can slow down in time. These small, timely messages are the practical value of the software.

5. System Requirements

Requirement	Minimum specification
Processor	Any dual-core processor (Intel / AMD / ARM)
RAM	512 MB

Requirement	Minimum specification
Storage	10 MB free (for the program and data files)
Operating system	Windows 10/11, Ubuntu / any Linux, macOS
Compiler	g++ (GCC) 5 or above, MinGW, Code::Blocks, Dev-C++, or Visual Studio (C++11 support)
Language / standard	C++11
Libraries used	Only the C++ Standard Library: iostream, fstream, sstream, vector, map, string, iomanip, ctime, algorithm

How to compile and run

```
g++ -std=c++11 ExpenseTracker.cpp -o tracker
./tracker           (Linux / macOS)
tracker.exe         (Windows)
```

The program creates three data files in the same folder on first use: **transactions.txt**, **budget.txt** and **challenge.txt**.

Menu of the application

Option	Function
1	Add Expense - date, amount, category, description; budget check runs immediately
2	Add Income - date, amount, source, description
3	View All Transactions in a table
4	Delete a transaction by ID
5	Search by keyword in category or description
6	Monthly Summary with category-wise bars and net balance
7	Set / Check Monthly Budget
8	**No-Spend Streak Challenge (unique feature)**
0	Save and Exit

6. OOP Concepts Used

The assignment is meant to test understanding of object oriented programming. The table below shows where each concept appears in the code, followed by a short explanation.

OOP concept	Where it is used in the project
Class and Object	Date, Transaction, Expense, Income, Budget, ExpenseTracker, StreakChallenge - objects are created in main()
Encapsulation	Data members are private/protected; accessed only through public member functions (getAmount(), setLimit() ...)
Abstraction	Transaction is an abstract class with pure virtual functions getType() and getLabel()
Inheritance	Expense and Income inherit from Transaction (single, public inheritance)
Polymorphism (run-time)	showAll() calls display()/serialize() through Transaction* pointers; the correct derived version runs
Virtual destructor	~Transaction() is virtual so derived objects are deleted correctly through the base pointer
Operator overloading	Date overloads ==, <, <=, + and - to compare dates and add/subtract days
Composition (has-a)	Transaction contains a Date; ExpenseTracker owns a collection of Transaction objects
Static members	Date::today(), Date::isLeap(), Date::daysInMonth(), StreakChallenge::badgeFor()
Constructors / Destructors	Default-argument constructors; ExpenseTracker destructor frees all dynamic memory
File handling	ifstream / ofstream used by save() and load() in each class
STL containers	vector<Transaction*> for storage, map for category totals and per-day totals

6.1 Explanation with examples

Abstraction and Inheritance

A *Transaction* is a general idea: it has an id, date, amount and description. Whether the money came in or went out is decided by the child class. Because Transaction has pure virtual functions, no object of Transaction itself can be created; only *Expense* and *Income* objects exist. This is abstraction - the base class defines the interface, and the derived classes provide the details.

```
class Transaction {
protected: int id; Date date; double amount; string description;
public:
    virtual string getType()  const = 0;   // pure virtual
    virtual string getLabel() const = 0;
    virtual ~Transaction() {}
};
class Expense : public Transaction { string category; ... };
class Income  : public Transaction { string source;   ... };
```

Polymorphism

The tracker stores all records in one `vector<Transaction*>`. When `list[i]->display()` is called, C++ decides at run time whether the object is an Expense or an Income and runs the correct function. The tracker does not need any if-else to find the type - this is polymorphism, and it makes the code easy to extend (for example a new *Transfer* class could be added without changing the tracker).

Encapsulation

The budget limit is private in the *Budget* class. Other code cannot set it to a negative number directly; it must call `setLimit()`, and the input functions ensure that only valid values reach it. In the same way *Date* keeps day, month and year private and offers `isValid()` to reject dates such as 31-02-2026.

Operator overloading

Dates are compared like numbers: `if (startDate <= today)`. `startDate + 5` gives the date five days later, and `today - startDate` gives the number of days between them. Internally a date is converted to the count of days since 01-01-1970, which also handles leap years. This is the foundation of the unique feature.

7. System Design and Class Description

The project is divided into seven classes. Each class has a single clear responsibility.

Class	Responsibility	Important members
Date	Stores a calendar date, validates it, compares and calculates with dates	day, month, year; today(), isValid(), toDays(), fromDays(), operators
Transaction (abstract)	Common data of every money record; defines the interface	id, date, amount, description; getType(), getLabel(), serialize(), display()
Expense	Money spent; adds a category	category
Income	Money received; adds a source	source
Budget	Monthly spending limit and warning logic	monthlyLimit; setLimit(), check(), save(), load()
ExpenseTracker	Manager class: owns all transactions and provides add / delete / search / summary / file operations	vector<Transaction*> list; addExpense(), remove(), search(), monthlySummary(), expenseOn(), hasExpenseOn()
StreakChallenge	**Unique feature**: analyses no-spend days and shows streak, savings, badge and calendar	startDate, targetDays, active; setup(), report(), calendar()

7.1 Relationships between classes

- **Inheritance (is-a):** Expense is-a Transaction; Income is-a Transaction.
- **Composition (has-a):** every Transaction has a Date; ExpenseTracker owns its Transaction objects and deletes them in its destructor.
- **Dependency (uses):** StreakChallenge uses ExpenseTracker (passed by const reference) to find out on which dates expenses exist. It never modifies the data.
- **Usage in main():** main() creates one ExpenseTracker, one Budget and one StreakChallenge, loads their files and runs the menu loop.

7.2 Overall working (flow)

1. Program starts and loads transactions.txt, budget.txt and challenge.txt (if present).
2. The main menu is displayed repeatedly until the user chooses 0.
3. Each menu choice calls a member function of the proper object.
4. When the user adds an expense, the monthly total is calculated and compared with the Budget immediately.
5. On exit, all three objects save their data back to files.

7.3 Design decisions

Why a pointer vector? A `vector<Expense>` and a `vector<Income>` would need duplicate code for every operation. A single `vector<Transaction*>` works for both through

polymorphism.

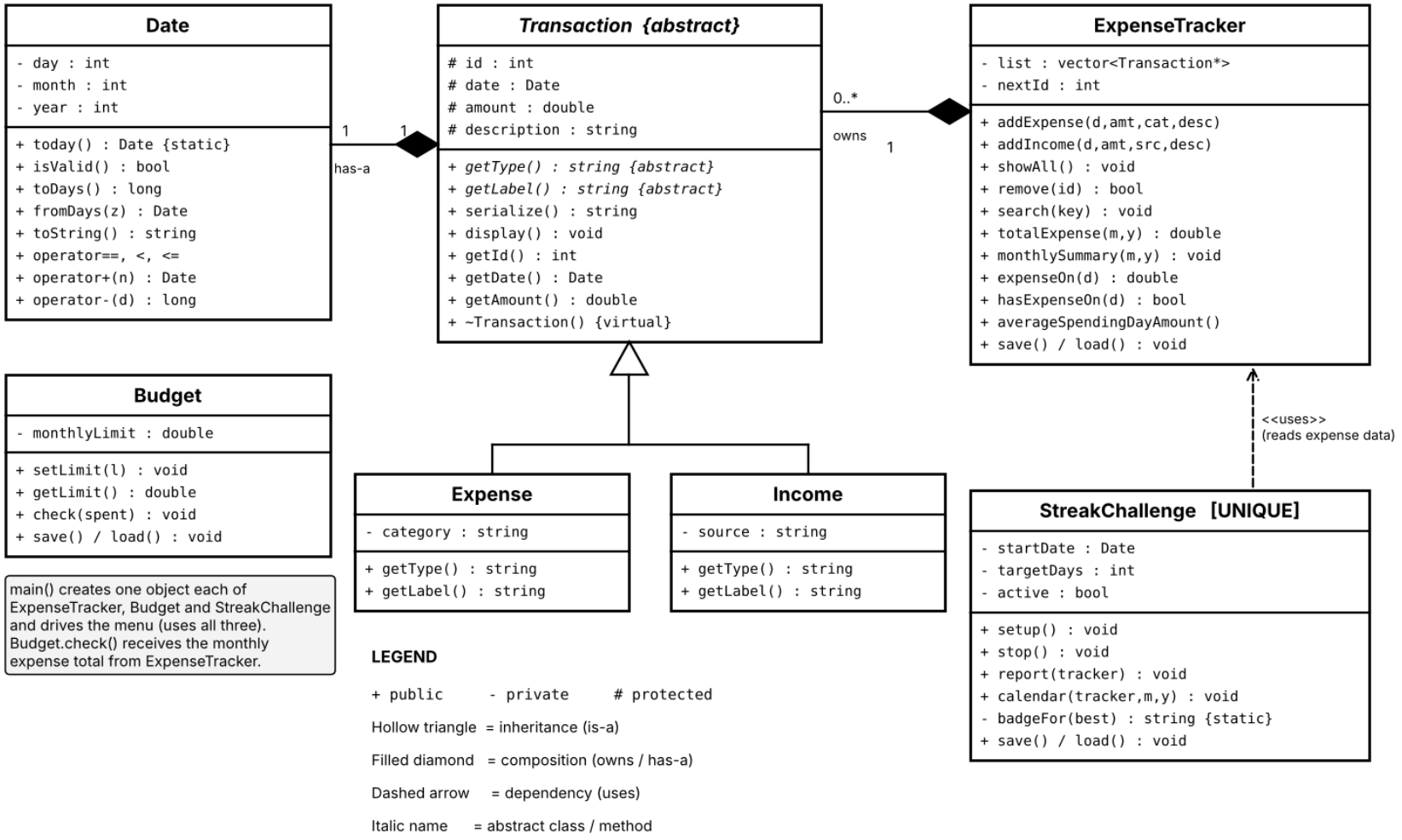
Why days-since-1970 for dates? Comparing day, month and year separately is error prone. Converting each date to one integer makes comparison, subtraction and looping over a range of days very simple and automatically correct for leap years.

Why a text file? A plain text file with the **|** separator can be opened in Notepad, inspected by the teacher and repaired by hand if needed. The program replaces any **|** typed by the user with **/** to protect the file format.

Why is the copy constructor of ExpenseTracker private? The class owns raw pointers. If the object were copied, two objects would delete the same memory (double free). Making copying impossible avoids this error completely.

Input validation: The helper functions readInt(), readDouble(), readDate() and readLine() keep asking until a valid value is entered. They also clear the error state of cin and discard bad characters, so typing letters where a number is expected does not cause an infinite loop.

8. UML Class Diagram



UML Class Diagram - Personal Expense Tracker

9. Unique Feature: No-Spend Streak Challenge

9.1 Idea

A normal expense tracker only *looks backwards*: it tells the user what was already spent. My unique feature makes the tracker *motivating*. The user starts a **challenge** (for example, "seven days in a row with zero spending") and the program turns the recorded data into a game with streaks, progress and badges. A day counts as a **no-spend day** if no expense is recorded on that date. Income does not break a streak.

9.2 What the feature does

- Starts a challenge from any date with a target streak between 1 and 365 days.
- Counts the total number of no-spend days since the start date.
- Calculates the **current streak** (consecutive no-spend days up to today) and the **best streak**.
- Shows the date of the **last spending day**, i.e. where the streak was broken.
- Estimates **money saved** = number of no-spend days x average amount spent on a spending day.
- Displays a **progress bar** towards the target and how many days are still needed.
- Awards a **badge** according to the best streak: Bronze (3+ days), Silver (7+), Gold (14+), Platinum (30+).
- Prints a **monthly calendar** where * marks a no-spend day, # a day with spending and . a day outside the challenge.
- Saves the challenge in challenge.txt so that it continues after restarting the program.

9.3 Algorithm

```
1. today  <- current date;  total <- today - startDate + 1
2. noSpend <- 0; run <- 0; best <- 0
3. FOR i = 0 TO total-1
       d <- startDate + i
       IF tracker.hasExpenseOn(d) = false THEN
            noSpend++ ; run++ ; best <- max(best, run)
       ELSE
            run <- 0 ; lastSpend <- d
4. current <- run              (streak that is still running today)
5. avg   <- average spending of the days that had expenses
6. saved <- noSpend * avg
7. progress <- min(100, current * 100 / targetDays)
8. badge <- badgeFor(best);  print the report
```

9.4 Implementation details

The feature is implemented in a separate class, **StreakChallenge**, so the original design stays clean. The class receives the *ExpenseTracker* as a **const reference**; this guarantees at compile time that the challenge can only read the data.

Date arithmetic. The loop `startDate + i` uses the overloaded `operator+` of Date. Behind it are two functions, `toDays()` and `fromDays()`, that convert a date to the number of days since 01-01-1970 and back. This makes the loop correct across month ends, year ends and leap years.

Calendar weekday. 01-01-1970 was a Thursday, so the weekday of any date is obtained as `(days + 3) mod 7` with Monday = 0. The first day of the month decides how many blank cells are printed before day 1.

Badge. A private static function `badgeFor(int best)` returns the badge name from the best streak.

9.5 How this feature is different from a normal tracker

Normal expense tracker	With No-Spend Streak Challenge
Shows only totals of money already spent	Rewards the days on which no money was spent
Passive record keeping	Active goal setting with target, progress bar and badges
No notion of habit	Tracks habit through current streak and best streak
Savings are not visible	Estimates the money saved by the no-spend days
Table of transactions only	Calendar view of good days (*) and spending days (#)

9.6 Usage

From the main menu select option **8**. Choose *Start new challenge*, enter the start date (or 0 for today) and the target number of days. Later choose *View my streak report* to see the analysis or *View monthly calendar* to see the month. Because the feature reads the normal expense records, the user does nothing extra: simply keep adding the real expenses and the streak updates itself.

10. File Handling and Data Format

Every transaction is written as one line with fields separated by the | character:

```
TYPE|ID|DAY|MONTH|YEAR|AMOUNT|CATEGORY-or-SOURCE|DESCRIPTION
EXPENSE|2|2|11|2026|150|Food|Canteen lunch
INCOME|1|1|11|2026|20000|Pocket money|Monthly allowance
```

budget.txt holds one number (the monthly limit). **challenge.txt** holds five values: active flag, start day, start month, start year and target days, for example `1 1 11 2026 5`. While loading, a line with fewer than eight fields is ignored, so a damaged line cannot crash the program.

11. Source Code (ExpenseTracker.cpp)

The complete, working program is given below. It compiles without errors or warnings with `g++ -std=c++11 -Wall -Wextra`.

```
/*
 * ============================================================
 *  PERSONAL EXPENSE TRACKER  -  OOP Project in C++
 * ------------------------------------------------------------
 *  Name        : Suman
 *  UID         : 25BCD10173
 *  Branch      : BCA (Data Science)    Section : 25BCD3 (B)
 *  Institute   : Chandigarh University
 *  Submitted to: Abhinay Sood
 * ------------------------------------------------------------
 *  UNIQUE FEATURE: "No-Spend Streak Challenge"
 *  The user starts a challenge (e.g. 14 days) and the program
 *  tracks how many consecutive days had ZERO expenses, the best
 *  streak, estimated money saved, a monthly calendar and badges.
 * ------------------------------------------------------------
 *  Compile : g++ -std=c++11 ExpenseTracker.cpp -o tracker
 *  Run     : ./tracker      (Windows: tracker.exe)
 * ============================================================
 */
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <map>
#include <string>
#include <iomanip>
#include <ctime>
#include <cstdlib>
#include <limits>
#include <algorithm>
using namespace std;

/* ---------- File names used for permanent storage ---------- */
const string DATA_FILE      = "transactions.txt";
const string BUDGET_FILE    = "budget.txt";
const string CHALLENGE_FILE = "challenge.txt";

/* ====================== CLASS: Date ======================== */
class Date {
private:
    int day, month, year;
public:
    Date(int d = 1, int m = 1, int y = 2000) : day(d), month(m), year(y) {}

    static Date today() {
        time_t t = time(0);
        tm* lt = localtime(&t);
        return Date(lt->tm_mday, lt->tm_mon + 1, lt->tm_year + 1900);
    }
    static bool isLeap(int y) { return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0); }
    static int daysInMonth(int m, int y) {
        static const int dm[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        if (m == 2 && isLeap(y)) return 29;
        return dm[m - 1];
    }
    bool isValid() const {
        return year >= 1970 && year <= 2100 && month >= 1 && month <= 12 &&
               day >= 1 && day <= daysInMonth(month, year);
    }
    int getDay() const { return day; }
    int getMonth() const { return month; }
    int getYear() const { return year; }

    /* days since 1970-01-01 (lets us do date arithmetic easily) */
    long toDays() const {
        int yy = year - (month <= 2 ? 1 : 0);
        long era = (yy >= 0 ? yy : yy - 399) / 400;
        long yoe = yy - era * 400;
        long doy = (153 * (month + (month > 2 ? -3 : 9)) + 2) / 5 + day - 1;
        long doe = yoe * 365 + yoe / 4 - yoe / 100 + doy;
        return era * 146097 + doe - 719468;
    }
    static Date fromDays(long z) {
        z += 719468;
        long era = (z >= 0 ? z : z - 146096) / 146097;
        long doe = z - era * 146097;
        long yoe = (doe - doe / 1460 + doe / 36524 - doe / 146096) / 365;
        long yy = yoe + era * 400;
        long doy = doe - (365 * yoe + yoe / 4 - yoe / 100);
        long mp = (5 * doy + 2) / 153;
        long dd = doy - (153 * mp + 2) / 5 + 1;
        long mm = mp < 10 ? mp + 3 : mp - 9;
        return Date((int)dd, (int)mm, (int)(yy + (mm <= 2 ? 1 : 0)));
    }
    string toString() const {
        ostringstream os;
```

```
        os << setfill('0') << setw(2) << day << "-" << setw(2) << month << "-" << year;
        return os.str();
    }
    /* operator overloading */
    bool operator==(const Date& o) const { return toDays() == o.toDays(); }
    bool operator<(const Date& o)  const { return toDays() <  o.toDays(); }
    bool operator<=(const Date& o) const { return toDays() <= o.toDays(); }
    Date operator+(int n) const { return fromDays(toDays() + n); }
    long operator-(const Date& o) const { return toDays() - o.toDays(); }
};

/* ============ CLASS: Transaction (abstract base) =========== */
class Transaction {
protected:
    int id;
    Date date;
    double amount;
    string description;
public:
    Transaction(int i, Date d, double a, string desc)
        : id(i), date(d), amount(a), description(desc) {}
    virtual ~Transaction() {}

    virtual string getType()  const = 0;   /* "EXPENSE" or "INCOME"      */
    virtual string getLabel() const = 0;   /* category  or source        */

    virtual string serialize() const {
        ostringstream os;
        os << getType() << "|" << id << "|" << date.getDay() << "|" << date.getMonth()
           << "|" << date.getYear() << "|" << amount << "|" << getLabel() << "|" << description;
        return os.str();
    }
    virtual void display() const {
        cout << left << setw(5) << id << setw(12) << date.toString()
             << setw(9) << getType() << setw(14) << getLabel()
             << right << setw(10) << fixed << setprecision(2) << amount
             << "  " << left << description << endl;
    }
    int    getId()     const { return id; }
    Date   getDate()   const { return date; }
    double getAmount() const { return amount; }
    string getDescription() const { return description; }
};

/* ================ CLASS: Expense ========================== */
class Expense : public Transaction {
private:
    string category;
public:
    Expense(int i, Date d, double a, string cat, string desc)
        : Transaction(i, d, a, desc), category(cat) {}
    string getType()  const { return "EXPENSE"; }
    string getLabel() const { return category; }
};

/* ================ CLASS: Income ========================== */
class Income : public Transaction {
private:
    string source;
public:
    Income(int i, Date d, double a, string src, string desc)
        : Transaction(i, d, a, desc), source(src) {}
    string getType()  const { return "INCOME"; }
    string getLabel() const { return source; }
};

/* ============ Small helper functions for input ============= */
string readLine(const string& prompt) {
    string s;
    while (true) {
        cout << prompt;
        getline(cin, s);
        if (!s.empty()) {
            for (size_t i = 0; i < s.size(); i++) if (s[i] == '|') s[i] = '/';
            return s;
        }
        cout << "  Input cannot be empty.\n";
    }
}
int readInt(const string& prompt, int lo, int hi) {
    int v;
    while (true) {
        cout << prompt;
        if (cin >> v && v >= lo && v <= hi) { cin.ignore(10000, '\n'); return v; }
        cin.clear(); cin.ignore(10000, '\n');
        cout << "  Enter a whole number between " << lo << " and " << hi << ".\n";
    }
}
double readDouble(const string& prompt, double minv) {
    double v;
    while (true) {
        cout << prompt;
        if (cin >> v && v >= minv) { cin.ignore(10000, '\n'); return v; }
        cin.clear(); cin.ignore(10000, '\n');
        cout << "  Enter a valid amount (minimum " << minv << ").\n";
```

```
    }
}
Date readDate(const string& prompt) {
    while (true) {
        cout << prompt << " (DD MM YYYY, or 0 for today): ";
        int d;
        if (!(cin >> d)) { cin.clear(); cin.ignore(10000, '\n'); cout << "  Invalid.\n"; continue; }
        if (d == 0) { cin.ignore(10000, '\n'); return Date::today(); }
        int m, y;
        if (!(cin >> m >> y)) { cin.clear(); cin.ignore(10000, '\n'); cout << "  Invalid.\n"; continue; }
        cin.ignore(10000, '\n');
        Date dt(d, m, y);
        if (dt.isValid()) return dt;
        cout << "  That date does not exist. Try again.\n";
    }
}
vector<string> split(const string& s, char sep) {
    vector<string> parts; string cur;
    for (size_t i = 0; i < s.size(); i++) {
        if (s[i] == sep) { parts.push_back(cur); cur.clear(); }
        else cur += s[i];
    }
    parts.push_back(cur);
    return parts;
}

/* ============== CLASS: Budget ============================== */
class Budget {
private:
    double monthlyLimit;
public:
    Budget() : monthlyLimit(0) {}
    void setLimit(double l) { monthlyLimit = l; }
    double getLimit() const { return monthlyLimit; }
    void save() const { ofstream f(BUDGET_FILE.c_str()); f << monthlyLimit << endl; }
    void load() { ifstream f(BUDGET_FILE.c_str()); if (f) f >> monthlyLimit; }
    void check(double spent) const {
        if (monthlyLimit <= 0) { cout << "  No budget set yet (use menu option 7).\n"; return; }
        double pct = spent * 100.0 / monthlyLimit;
        cout << "  Budget: " << fixed << setprecision(2) << monthlyLimit
             << "   Spent: " << spent << "   (" << setprecision(1) << pct << "%)\n";
        if (spent > monthlyLimit)      cout << "  *** ALERT: You have EXCEEDED your budget by "
                                            << setprecision(2) << spent - monthlyLimit << " ***\n";
        else if (pct >= 80)            cout << "  WARNING: More than 80% of the budget is used.\n";
        else                           cout << "  Good: You are within budget. Remaining "
                                            << setprecision(2) << monthlyLimit - spent << "\n";
    }
};

/* ============== CLASS: ExpenseTracker (manager) ============ */
class ExpenseTracker {
private:
    vector<Transaction*> list;
    int nextId;
    ExpenseTracker(const ExpenseTracker&);             /* no copying */
    ExpenseTracker& operator=(const ExpenseTracker&);
public:
    ExpenseTracker() : nextId(1) {}
    ~ExpenseTracker() { for (size_t i = 0; i < list.size(); i++) delete list[i]; }

    void addExpense(Date d, double amt, const string& cat, const string& desc) {
        list.push_back(new Expense(nextId++, d, amt, cat, desc));
    }
    void addIncome(Date d, double amt, const string& src, const string& desc) {
        list.push_back(new Income(nextId++, d, amt, src, desc));
    }
    void showAll() const {
        if (list.empty()) { cout << "  No transactions yet.\n"; return; }
        cout << left << setw(5) << "ID" << setw(12) << "Date" << setw(9) << "Type"
             << setw(14) << "Category" << right << setw(10) << "Amount" << "  Description\n";
        cout << string(70, '-') << endl;
        for (size_t i = 0; i < list.size(); i++) list[i]->display();   /* polymorphism */
    }
    bool remove(int id) {
        for (size_t i = 0; i < list.size(); i++)
            if (list[i]->getId() == id) { delete list[i]; list.erase(list.begin() + i); return true; }
        return false;
    }
    void search(const string& key) const {
        string k = key; transform(k.begin(), k.end(), k.begin(), ::tolower);
        int found = 0;
        for (size_t i = 0; i < list.size(); i++) {
            string a = list[i]->getLabel() + " " + list[i]->getDescription();
            transform(a.begin(), a.end(), a.begin(), ::tolower);
            if (a.find(k) != string::npos) { list[i]->display(); found++; }
        }
        if (!found) cout << "  No matching records.\n";
    }
    double totalExpense(int month, int year) const {
        double t = 0;
        for (size_t i = 0; i < list.size(); i++)
            if (list[i]->getType() == "EXPENSE" && list[i]->getDate().getMonth() == month &&
                list[i]->getDate().getYear() == year) t += list[i]->getAmount();
        return t;
    }
```

```
    void monthlySummary(int month, int year) const {
        map<string, double> cat; double inc = 0, exp = 0;
        for (size_t i = 0; i < list.size(); i++) {
            Date d = list[i]->getDate();
            if (d.getMonth() != month || d.getYear() != year) continue;
            if (list[i]->getType() == "EXPENSE") { cat[list[i]->getLabel()] += list[i]->getAmount(); exp += list[i]->getAmount(); }
            else inc += list[i]->getAmount();
        }
        cout << "\n  ---- Summary for " << month << "/" << year << " ----\n";
        for (map<string, double>::iterator it = cat.begin(); it != cat.end(); ++it) {
            int bars = exp > 0 ? (int)(it->second * 30 / exp) : 0;
            cout << "  " << left << setw(14) << it->first << right << setw(10) << fixed << setprecision(2)
                 << it->second << "  " << string(bars, '#') << endl;
        }
        cout << "  -------------------------------------\n";
        cout << "  Total Income  : " << inc << "\n  Total Expense : " << exp
             << "\n  Net Balance   : " << inc - exp << endl;
    }
    /* total expense on one particular day (used by the unique feature) */
    double expenseOn(const Date& d) const {
        double t = 0;
        for (size_t i = 0; i < list.size(); i++)
            if (list[i]->getType() == "EXPENSE" && list[i]->getDate() == d) t += list[i]->getAmount();
        return t;
    }
    bool hasExpenseOn(const Date& d) const {
        for (size_t i = 0; i < list.size(); i++)
            if (list[i]->getType() == "EXPENSE" && list[i]->getDate() == d) return true;
        return false;
    }
    double averageSpendingDayAmount(const Date& from, const Date& to) const {
        map<long, double> perDay;
        for (size_t i = 0; i < list.size(); i++)
            if (list[i]->getType() == "EXPENSE" && from <= list[i]->getDate() && list[i]->getDate() <= to)
                perDay[list[i]->getDate().toDays()] += list[i]->getAmount();
        if (perDay.empty()) return 0;
        double sum = 0;
        for (map<long, double>::iterator it = perDay.begin(); it != perDay.end(); ++it) sum += it->second;
        return sum / perDay.size();
    }
    void save() const {
        ofstream f(DATA_FILE.c_str());
        for (size_t i = 0; i < list.size(); i++) f << list[i]->serialize() << "\n";
    }
    void load() {
        ifstream f(DATA_FILE.c_str());
        string line;
        while (getline(f, line)) {
            vector<string> p = split(line, '|');
            if (p.size() < 8) continue;
            int id = atoi(p[1].c_str());
            Date d(atoi(p[2].c_str()), atoi(p[3].c_str()), atoi(p[4].c_str()));
            double amt = atof(p[5].c_str());
            if (p[0] == "EXPENSE") list.push_back(new Expense(id, d, amt, p[6], p[7]));
            else                   list.push_back(new Income(id, d, amt, p[6], p[7]));
            if (id >= nextId) nextId = id + 1;
        }
    }
};

/* ============================================================
 *  CLASS: StreakChallenge   <<< MY UNIQUE FEATURE >>>
 *  "No-Spend Streak Challenge": rewards the user for days on
 *  which no expense was recorded.
 * ============================================================ */
class StreakChallenge {
private:
    Date startDate;
    int  targetDays;
    bool active;

    static string badgeFor(int best) {
        if (best >= 30) return "PLATINUM  (30+ day streak)";
        if (best >= 14) return "GOLD      (14+ day streak)";
        if (best >= 7)  return "SILVER    (7+ day streak)";
        if (best >= 3)  return "BRONZE    (3+ day streak)";
        return "No badge yet - keep going!";
    }
public:
    StreakChallenge() : startDate(Date::today()), targetDays(7), active(false) {}

    bool isActive() const { return active; }

    void setup() {
        startDate  = readDate("  Challenge start date");
        targetDays = readInt("  Target streak in days (1-365): ", 1, 365);
        active = true;
        save();
        cout << "  Challenge started on " << startDate.toString() << "! Goal: " << targetDays
             << " no-spend days in a row.\n";
    }
    void stop() { active = false; save(); cout << "  Challenge stopped.\n"; }

    void save() const {
        ofstream f(CHALLENGE_FILE.c_str());
```

```
        f << active << " " << startDate.getDay() << " " << startDate.getMonth() << " "
          << startDate.getYear() << " " << targetDays << endl;
    }
    void load() {
        ifstream f(CHALLENGE_FILE.c_str());
        int a, d, m, y, t;
        if (f >> a >> d >> m >> y >> t) {
            Date dt(d, m, y);
            if (dt.isValid()) { active = (a == 1); startDate = dt; targetDays = t; }
        }
    }

    /* Main analysis report */
    void report(const ExpenseTracker& tr) const {
        if (!active) { cout << "  No active challenge. Choose 'Start new challenge' first.\n"; return; }
        Date today = Date::today();
        if (today < startDate) { cout << "  Challenge starts in the future (" << startDate.toString() << ").\n"; return; }

        long total = today - startDate + 1;
        int noSpend = 0, run = 0, best = 0, current = 0;
        Date brokenOn = startDate; bool everBroken = false;
        for (long i = 0; i < total; i++) {
            Date d = startDate + (int)i;
            if (!tr.hasExpenseOn(d)) { noSpend++; run++; if (run > best) best = run; }
            else { run = 0; brokenOn = d; everBroken = true; }
        }
        current = run;                                  /* streak ending today */
        double avg   = tr.averageSpendingDayAmount(startDate, today);
        double saved = avg * noSpend;
        double pct   = current * 100.0 / targetDays; if (pct > 100) pct = 100;

        cout << "\n  ======== NO-SPEND STREAK CHALLENGE ========\n";
        cout << "  Started        : " << startDate.toString() << "   (" << total << " days so far)\n";
        cout << "  Target         : " << targetDays << " days in a row\n";
        cout << "  No-spend days  : " << noSpend << " / " << total << "\n";
        cout << "  Current streak : " << current << " day(s)\n";
        cout << "  Best streak    : " << best << " day(s)\n";
        if (everBroken) cout << "  Last spend day : " << brokenOn.toString() << "\n";
        cout << "  Est. money saved: " << fixed << setprecision(2) << saved
             << "  (no-spend days x avg daily spend " << avg << ")\n";
        int filled = (int)(pct / 5);
        cout << "  Progress       : [" << string(filled, '=') << string(20 - filled, ' ') << "] "
             << setprecision(0) << pct << "%\n";
        cout << "  Badge earned   : " << badgeFor(best) << "\n";
        if (current >= targetDays) cout << "  *** CONGRATULATIONS! Challenge goal achieved! ***\n";
        else cout << "  " << targetDays - current << " more no-spend day(s) to reach the goal.\n";
    }

    /* Monthly calendar: * = no-spend day, # = spent money, . = outside challenge */
    void calendar(const ExpenseTracker& tr, int month, int year) const {
        if (!active) { cout << "  No active challenge.\n"; return; }
        static const char* names[] = {"January","February","March","April","May","June","July",
                                      "August","September","October","November","December"};
        cout << "\n        " << names[month - 1] << " " << year << "\n";
        cout << " Mo   Tu   We   Th   Fr   Sa   Su\n" << right;
        Date first(1, month, year);
        int wd = (int)(((first.toDays() + 3) % 7 + 7) % 7);   /* 0 = Monday (1970-01-01 was Thursday) */
        cout << string(wd * 5, ' ');
        Date today = Date::today();
        for (int d = 1; d <= Date::daysInMonth(month, year); d++) {
            Date cur(d, month, year);
            char mark = '.';
            if (startDate <= cur && cur <= today) mark = tr.hasExpenseOn(cur) ? '#' : '*';
            cout << setw(3) << d << mark << " ";
            if ((wd + d) % 7 == 0) cout << "\n";
        }
        cout << "\n   Legend:  * = no-spend day    # = spent money    . = not in challenge\n";
    }
};

/* ====================== MENUS ============================== */
void streakMenu(StreakChallenge& sc, const ExpenseTracker& tr) {
    while (true) {
        cout << "\n  --- NO-SPEND STREAK CHALLENGE ---\n"
             << "  1. Start new challenge\n  2. View my streak report\n"
             << "  3. View monthly calendar\n  4. Stop challenge\n  0. Back\n";
        int c = readInt("  Choice: ", 0, 4);
        if (c == 0) return;
        if (c == 1) sc.setup();
        else if (c == 2) sc.report(tr);
        else if (c == 3) {
            int m = readInt("  Month (1-12): ", 1, 12);
            int y = readInt("  Year: ", 1970, 2100);
            sc.calendar(tr, m, y);
        } else sc.stop();
    }
}

int main() {
    ExpenseTracker tracker;
    Budget budget;
    StreakChallenge challenge;
    tracker.load(); budget.load(); challenge.load();

    cout << "=========================================\n"
```

```
         << "      PERSONAL EXPENSE TRACKER\n"
         << "  Chandigarh University | BCA Data Science\n"
         << "========================================\n";
    while (true) {
        cout << "\n 1. Add Expense\n 2. Add Income\n 3. View All Transactions\n"
             << " 4. Delete Transaction\n 5. Search\n 6. Monthly Summary\n"
             << " 7. Set / Check Monthly Budget\n 8. No-Spend Streak Challenge  [NEW]\n"
             << " 0. Save & Exit\n";
        int ch = readInt(" Enter choice: ", 0, 8);
        if (ch == 0) break;
        switch (ch) {
        case 1: {
            Date d = readDate("  Date");
            double a = readDouble("  Amount: ", 0.01);
            string cat = readLine("  Category (Food/Travel/Rent/Shopping...): ");
            string desc = readLine("  Description: ");
            tracker.addExpense(d, a, cat, desc);
            cout << "  Expense added.\n";
            budget.check(tracker.totalExpense(d.getMonth(), d.getYear()));
            break; }
        case 2: {
            Date d = readDate("  Date");
            double a = readDouble("  Amount: ", 0.01);
            string src = readLine("  Source (Salary/Pocket money/Gift...): ");
            string desc = readLine("  Description: ");
            tracker.addIncome(d, a, src, desc);
            cout << "  Income added.\n";
            break; }
        case 3: tracker.showAll(); break;
        case 4: {
            int id = readInt("  Enter ID to delete: ", 1, 1000000);
            cout << (tracker.remove(id) ? "  Deleted.\n" : "  ID not found.\n");
            break; }
        case 5: tracker.search(readLine("  Keyword: ")); break;
        case 6: {
            int m = readInt("  Month (1-12): ", 1, 12);
            int y = readInt("  Year: ", 1970, 2100);
            tracker.monthlySummary(m, y);
            budget.check(tracker.totalExpense(m, y));
            break; }
        case 7: {
            cout << "  Current limit: " << budget.getLimit() << "\n";
            if (readInt("  1 = change limit, 2 = just check this month: ", 1, 2) == 1)
                budget.setLimit(readDouble("  New monthly limit: ", 1));
            Date t = Date::today();
            budget.check(tracker.totalExpense(t.getMonth(), t.getYear()));
            break; }
        case 8: streakMenu(challenge, tracker); break;
        }
    }
    tracker.save(); budget.save(); challenge.save();
    cout << "  Data saved. Goodbye!\n";
    return 0;
}
```

12. Sample Output

12.1 Main menu

```
========================================
      PERSONAL EXPENSE TRACKER
  Chandigarh University | BCA Data Science
========================================

 1. Add Expense
 2. Add Income
 3. View All Transactions
 4. Delete Transaction
 5. Search
 6. Monthly Summary
 7. Set / Check Monthly Budget
 8. No-Spend Streak Challenge  [NEW]
 0. Save & Exit
```

12.2 View all transactions (option 3)

```
 ID   Date        Type     Category          Amount  Description
-------------------------------------------------------------------
1    01-11-2026  INCOME   Pocket money    20000.00  Monthly allowance
2    02-11-2026  EXPENSE  Food              150.00  Canteen lunch
3    04-11-2026  EXPENSE  Shopping          420.00  Notebooks and pens
4    06-11-2026  EXPENSE  Travel            900.00  Bus pass recharge
5    09-11-2026  EXPENSE  Entertainment     250.00  Movie ticket
```

12.3 Monthly summary with budget check (option 6)

```
  ---- Summary for 11/2026 ----
  Entertainment     250.00  ####
  Food              150.00  ##
  Shopping          420.00  #######
  Travel            900.00  ################
  ------------------------------------
  Total Income  : 20000.00
  Total Expense : 1720.00
  Net Balance   : 18280.00
  Budget: 2500.00   Spent: 1720.00   (68.8%)
```

12.4 No-Spend Streak report - unique feature (option 8 - 2)

```
  ======== NO-SPEND STREAK CHALLENGE ========
  Started        : 01-11-2026   (10 days so far)
  Target         : 5 days in a row
  No-spend days  : 6 / 10
  Current streak : 1 day(s)
  Best streak    : 2 day(s)
  Last spend day : 09-11-2026
  Est. money saved: 2580.00  (no-spend days x avg daily spend 430.00)
  Progress       : [====                ] 20%
  Badge earned   : No badge yet - keep going!
  4 more no-spend day(s) to reach the goal.
```

12.5 Monthly calendar - unique feature (option 8 - 3)

```
        November 2026
 Mo   Tu   We   Th   Fr   Sa   Su
                                 1*
  2#   3*   4#   5*   6#   7*   8*
  9#  10*  11.  12.  13.  14.  15.
 16.  17.  18.  19.  20.  21.  22.
 23.  24.  25.  26.  27.  28.  29.
 30.
   Legend:  * = no-spend day    # = spent money    . = not in challenge
```

In the calendar the days 1, 3, 5, 7, 8 and 10 are no-spend days (*), while 2, 4, 6 and 9 are days on which an expense was recorded (#). The last spend day was 09-11-2026, so the current streak restarted after that date.

13. Testing

The program was tested with normal, boundary and wrong inputs. The table lists the main test cases and the result.

No.	Test case	Expected result	Result
1	Add expense with valid data	Record saved, budget message shown	Pass
2	Enter date 31 02 2026	"That date does not exist" - asks again	Pass
3	Type letters at the menu prompt	Message and re-prompt, no infinite loop	Pass
4	Enter amount -5	Rejected: minimum 0.01	Pass
5	Empty category / description	"Input cannot be empty"	Pass
6	Delete an ID that does not exist	"ID not found"	Pass
7	Delete a valid ID	Record removed from list and file	Pass
8	Search keyword in capitals / small letters	Case-insensitive match	Pass
9	Spending above the budget	ALERT with exceeded amount	Pass
10	Restart program	Transactions, budget, challenge reloaded	Pass
11	Streak: no expense since start	Current = best = number of days	Pass
12	Streak: expense on today's date	Current streak = 0	Pass
13	Challenge start date in the future	"Challenge starts in the future"	Pass
14	Calendar for a month with 28/29/30/31 days	Correct weekday alignment	Pass
15	Target reached (current >= target)	Congratulation message shown	Pass

Testing environment

Compiler: g++ (GCC) with flags `-std=c++11 -Wall -Wextra` - zero warnings. Operating system: Linux. The weekday calculation was verified: 01-11-2026 correctly appears as a Sunday in the calendar.

14. Limitations and Future Scope

Limitations

- Console interface only; there is no graphical user interface.
- A single user; no password protection of the data files.
- Data is stored in plain text files, not in a database.
- The streak counts a day as no-spend only if no expense is recorded; if the user forgets to enter an expense, the day is counted as no-spend.

Future scope

- Convert the project to a GUI application using Qt or a mobile app.
- Store data in SQLite and add user login with an encrypted password.
- Add category-wise budgets and export reports to CSV / Excel.
- Show charts (pie / bar) and monthly comparison graphs.
- Extend the streak feature with weekly challenges, a "streak freeze" day, and reminders.
- Add a new derived class *Transfer* (between accounts) - possible without changing ExpenseTracker because of polymorphism.

15. Conclusion

The Personal Expense Tracker fulfils all the requirements of the assignment: it is a working C++ program that has been compiled and run, it is designed with classes, inheritance, polymorphism, encapsulation, operator overloading and file handling, and it is documented with a UML class diagram and this report. The unique feature, the **No-Spend Streak Challenge**, changes the project from a simple record book into a tool that actively helps the user to save money.

While working on the project I learned how to divide a problem into classes, how a base-class pointer gives flexible code, how to handle input errors, how dates can be calculated without external libraries, and how to test a program systematically.

References

- Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.
- E. Balagurusamy, *Object Oriented Programming with C++*, McGraw Hill Education.
- cppreference.com - C++ Standard Library documentation.
- Howard Hinnant, *chrono-Compatible Low-Level Date Algorithms* (days-from-civil method used for date conversion).
- Lecture notes and lab material, Chandigarh University - Object Oriented Programming.